

Enterprise Analytics Hackathon

Project Documentation Report

AdventureWorks

Imama Zubair

AI INTERN @ NETIXSOL

Table of Contents

1. Introduction
2. Project Objectives
3. Scope
4. Tools and Technologies Used
5. Database Overview
6. Methodology
7. Implementation
8. Analysis
9. Findings
10. Challenges Faced
11. Screenshots
12. Conclusion

1. Introduction

This report documents the design and implementation of a reusable analytics layer built on top of the AdventureWorks operational database. The goal of the hackathon was to move away from writing isolated SQL queries against raw operational tables, and instead build a complete pipeline that transforms raw transactional data into reusable, business-ready analytical datasets that support executive reporting and dashboarding.

The project covers the full pipeline requested in the brief: an intermediate analytics layer built from the raw operational schema, a chained set of business metric views built on top of that layer, an executive KPI summary, a Python notebook that turns those views into visualizations and business insights, and a set of executive recommendations grounded directly in the analysis.

2. Project Objectives

- Design a reusable analytics layer on top of the AdventureWorks operational database so that raw tables are not queried repeatedly.
- Cover at least five different business domains and create at least ten reusable analytical tables or views.
- Build a chained SQL pipeline where each stage builds on the stage before it, rather than isolated queries.
- Produce dashboard-ready executive KPI datasets across sales, customers, products, employees, territories, and inventory/purchasing.
- Demonstrate advanced SQL concepts including window functions, CASE WHEN logic, conditional aggregation, ranking functions, and complex joins.
- Build a Python notebook that reads only from the analytics layer, visualizes the results, and presents business insights.
- Produce executive recommendations, covering opportunities, risks, and actionable next steps, each grounded in the SQL analysis.

3. Scope

The scope of this project covers the full AdventureWorks dataset (68 tables across 5 schemas), the analytics layer built on top of it, the executive analysis notebook, and the supporting documentation. The project does not modify or extend the raw operational schema itself, all analytical work is additive, built entirely in the analytics and kpi schemas alongside the untouched original data.

4. Tools and Technologies Used

1. PostgreSQL
2. pgAdmin
3. Jupyter Notebook
4. Python

5. Database Overview

The project uses the AdventureWorks sample database, the same sample database Microsoft ships for SQL Server, ported to PostgreSQL. It is a genuine enterprise schema with 68 tables spread across 5 schemas: person, humanresources, production, purchasing, and sales.

It has:

- 31,465 sales orders and 121,317 order lines
- 19,820 customers, of which 19,119 are individual customers and 701 are store or B2B accounts
- 504 products across 4 categories
- 290 employees, 17 of whom are salespeople
- 104 vendors and 4,012 purchase orders

6. Methodology

The project was built and verified in a strict order, testing each stage against the live database before moving to the next, rather than writing the full pipeline blind.

- Loaded the full AdventureWorks dataset using the official install script and verified row counts against expected totals for every table.
- Designed and built the ten Stage 1 domain tables, testing each one individually before building the next.
- Designed and built the twenty seven Stage 2 business metric views, reusing shared base aggregations where more than one downstream view needed the same calculation.
- Built the Stage 3 executive summary view chaining across six Stage 2 views.
- Built the Python notebook, executed it end to end against a live copy of the database, and verified every chart rendered correctly.

- Reviewed the resulting charts and numbers to write the executive recommendations, each one tied back to a specific view.

7. Implementation

The analytics layer contains 10 analytics tables, 27 KPI views, and one executive summary view. Shared views reduce repeated calculations while PostgreSQL schemas keep raw and analytics data separate

Stage 1 (analytics schema), 10 tables

Table	Business Domain	What It Covers
date_analytics	Generated	Calendar dimension spanning the full order history, with year, quarter, month, and fiscal year
product_analytics	Production	Product catalog with category, margin, and current inventory
territory_analytics	Sales	Sales territories with current salesperson and country/state rollup
vendor_analytics	Purchasing	Vendors with product count supplied and average lead time
employee_analytics	Human Resources	All employees with department, tenure, and salesperson flag
customer_analytics	Sales	Every customer, individual or store, with territory
sales_analytics	Sales	Order line grain, with margin computed using cost at time of sale
purchasing_analytics	Purchasing	Purchase order line grain, with lead time and rejection rate
inventory_analytics	Production	Current stock level against reorder point per product
geography_analytics	Person	Address, city, and state counts by country

Stage 2 (kpi schema): 27 views

Sales: monthly_revenue, quarterly_revenue, sales_growth, best_selling_products, lowest_performing_products, and the shared base product_sales_summary.

Products: product_rankings, product_profitability, category_performance. Employees: salesperson_sales_summary, salesperson_rankings, revenue_contribution, performance_comparison.

Territories: territory_sales_summary, regional_revenue, regional_growth, top_territories, lowest_territories. Customers: customer_metrics, customer_segments, customer_ltv, repeat_customers, customer_retention. Inventory and Purchasing: inventory_health, low_stock_products, supplier_performance, purchasing_trends.

Stage 3 (kpi schema):1 view

kpi.executive_summary.

8. Analysis

executive_analysis.ipynb connects to PostgreSQL using SQLAlchemy and psycopg2, and reads every dataset through pandas.read_sql() against the analytics and kpi schemas only. The notebook contains eight visualizations: revenue trend, sales by territory, customer segments, product performance, category revenue, employee performance, inventory status, and the executive KPI summary. Each chart is followed by a short written insight explaining what the business can learn from it.

9. Findings

Opportunities

- Repeat customers make up only 37 percent of the customer base by count but generate 93.8 percent of total revenue (\$103.1M of \$109.8M). Even a small increase in the rate at which one-time buyers become repeat buyers would have an outsized effect on revenue.
- Accessories carry a 55.3 percent average margin, the highest of any category, but only bring in \$1.27M in revenue compared to \$94.65M from Bikes. Selling more accessories alongside bike purchases is a low-risk way to lift overall margin.
- Southwest, Canada, and Northwest together generate 51.5 percent of all territory revenue. The remaining seven territories have real room to grow.
- International territories (United Kingdom, France, Germany, Australia) already contribute about 27.8 percent of total revenue combined, showing international demand is proven rather than speculative.
- Every salesperson with an assigned quota is currently exceeding it, suggesting either a strong sales team or quotas set too conservatively.

Risks

- All 701 store-channel (B2B) customers show zero direct orders in the sales data, either a real gap in that channel or a tracking issue that needs investigating.
- 80 of 504 products (15.9 percent) are currently at or below their reorder point, a meaningful share of the catalog at risk of stockouts.
- 209 of 504 products (about 41 percent) have no category assigned in the source data, understating any category level total until cleaned up.
- The most recent year of data (2025) only runs through June, so any year over year comparison involving 2025 will look worse than it really is unless this is accounted for.
- The worst-performing suppliers have rejection rates close to 6 percent, noticeably higher than the best performers, a quality risk if they supply high-volume products.

Recommendations

- Run a retention campaign aimed at the 11,649 One-Time customers, since repeat customers already drive almost all revenue.
- Bundle accessories with bike purchases at checkout to grow margin without needing new products or new customers.
- Turn the approach used in Southwest, Canada, and Northwest into a playbook for the remaining seven territories.
- Review and reset salesperson quotas for next year, since every current quota is being exceeded.
- Prioritize restocking the 80 low-stock products, starting with the highest revenue ones, and reduce order volume with suppliers whose rejection rates sit above 5 percent.

10. Challenges faced

1. The first dataset had a broken delimiter in 13 CSV files, which we only caught by actually running the load and checking row counts against expected totals not by reading the files by eye.
2. Several columns we expected to read directly (`LineTotal`, `StockedQty`, and
3. others) turned out to be dropped by the load script since they were computed columns in the original schema. Every view that needed them recomputes them.
4. Our first version of `territory\_analytics` returned 13 rows instead of 10, because three territories had two "current" salesperson history records nstead of one. Fixed with a `LATERAL` join that picks exactly one per territory.
5. 2025 is a partial year in this dataset (data runs through June 2025 only), so any year over year comparison involving 2025 needs a caveat or it reads as a real decline when it isn't one.

11. Screenshots

1: Database Created

Shows the AdventureWorks database in pgAdmin with all five schemas (person, humanresources, production, purchasing, sales)

The screenshot displays the pgAdmin interface. The top pane shows a SQL query with 12 lines, each preceded by a line number. The query is a UNION ALL of 12 SELECT statements, each counting the number of rows in a specific table. The bottom pane shows the results of the query in a table format with 12 rows and 2 columns: 'tbl' and 'count'. The 'tbl' column lists the table names, and the 'count' column shows the row counts. The status bar at the bottom indicates 'Total rows: 12' and 'Query complete 00:00:01.101'.

```
1 SELECT 'sales.salesorderheader' AS tbl, COUNT(*) FROM sales.salesorderheader
2 UNION ALL SELECT 'sales.salesorderdetail', COUNT(*) FROM sales.salesorderdetail
3 UNION ALL SELECT 'sales.customer', COUNT(*) FROM sales.customer
4 UNION ALL SELECT 'sales.salesperson', COUNT(*) FROM sales.salesperson
5 UNION ALL SELECT 'sales.salesterritory', COUNT(*) FROM sales.salesterritory
6 UNION ALL SELECT 'sales.store', COUNT(*) FROM sales.store
7 UNION ALL SELECT 'production.product', COUNT(*) FROM production.product
8 UNION ALL SELECT 'production.productinventory', COUNT(*) FROM production.productinventory
9 UNION ALL SELECT 'humanresources.employee', COUNT(*) FROM humanresources.employee
10 UNION ALL SELECT 'purchasing.vendor', COUNT(*) FROM purchasing.vendor
11 UNION ALL SELECT 'purchasing.purchaseorderheader', COUNT(*) FROM purchasing.purchaseorderheader
12 UNION ALL SELECT 'purchasing.purchaseorderdetail', COUNT(*) FROM purchasing.purchaseorderdetail;
```

	tbl	count
1	sales.salesterritory	10
2	sales.salesperson	17
3	purchasing.vendor	104
4	humanresources.employee	290
5	production.product	504
6	production.productinventory	1069
7	sales.store	701
8	purchasing.purchaseorderhe...	4012
9	purchasing.purchaseorderdet...	8845
10	sales.customer	19820
11	sales.salesorderheader	31465
12	sales.salesorderdetail	121317

Total rows: 12 Query complete 00:00:01.101

2: A Successfully Run View

Shows one of the Stage 1 analytics views (analytics.sales_analytics) executed


```

281 -- CHECKING SALES ANALYTICS
282 SELECT * FROM analytics.sales_analytics ORDER BY orderdate DESC LIMIT 10;

```

	salesorderid integer	salesorderdetailid integer	orderdate timestamp without time zone	customerid integer	salespersonid integer	territoryid integer	status smallint	productid integer	orderqty smallint	unitprice numeric	unitpricediscount numeric	line_total numeric	line_cost numeric	line_profit numeric
1	75088	121227	2025-06-29 00:00:00	14680	[null]	9	5	922	1	3.9900	0.0000	3.99	1.49	2.50
2	75088	121230	2025-06-29 00:00:00	14680	[null]	9	5	713	1	49.9900	0.0000	49.99	38.49	11.50
3	75085	121224	2025-06-29 00:00:00	11927	[null]	1	5	877	1	7.9500	0.0000	7.95	2.97	4.98
4	75087	121226	2025-06-29 00:00:00	11794	[null]	4	5	707	1	34.9900	0.0000	34.99	13.09	21.90
5	75088	121228	2025-06-29 00:00:00	14680	[null]	9	5	932	1	24.9900	0.0000	24.99	9.35	15.64
6	75088	121229	2025-06-29 00:00:00	14680	[null]	9	5	711	1	34.9900	0.0000	34.99	13.09	21.90
7	75084	121222	2025-06-29 00:00:00	11078	[null]	6	5	876	1	120.0000	0.0000	120.00	44.88	75.12
8	75085	121223	2025-06-29 00:00:00	11927	[null]	1	5	712	1	8.9900	0.0000	8.99	6.92	2.07
9	75086	121225	2025-06-29 00:00:00	28789	[null]	4	5	877	1	7.9500	0.0000	7.95	2.97	4.98
10	75089	121231	2025-06-29 00:00:00	19585	[null]	9	5	931	1	21.4900	0.0000	21.49	8.04	13.45

3: KPI Result of best selling products

Shows one of the Stage 2 business metric views (for example, kpi.best_selling_products or kpi.category_performance) executed successfully with results displayed.

```

690 --TASK 3 B: BEST SELLING PRODUCTS
691 SELECT * FROM kpi.best_selling_products;

```

	productid integer	product_name character varying (50)	category_name character varying (50)	total_qty_sold bigint	total_revenue numeric	revenue_rank bigint
1	782	Mountain-200 Black, 38	Bikes	2977	4400593.46	1
2	783	Mountain-200 Black, 42	Bikes	2664	4009495.59	2
3	779	Mountain-200 Silver, 38	Bikes	2394	3693678.11	3
4	780	Mountain-200 Silver, 42	Bikes	2234	3438478.85	4
5	781	Mountain-200 Silver, 46	Bikes	2216	3434256.97	5
6	784	Mountain-200 Black, 46	Bikes	2111	3309673.85	6
7	793	Road-250 Black, 44	Bikes	1642	2516857.34	7
8	794	Road-250 Black, 48	Bikes	1498	2347655.99	8
9	795	Road-250 Black, 52	Bikes	1245	2012447.65	9
10	753	Road-150 Red, 56	Bikes	664	1847818.36	10

4: Executive Summary

```

744 kpi.executive_summary
745 SELECT territory_name, lifetime_revenue FROM kpi.top_territories ORDER BY revenue_rank LIMIT 1
746 ),
747 top_product AS (
748 SELECT product_name, total_revenue FROM kpi.best_selling_products ORDER BY revenue_rank LIMIT 1
749 ),
750 top_salesperson AS (
751 SELECT employee_name, total_revenue FROM kpi.salesperson_rankings ORDER BY revenue_rank LIMIT 1
752 ),
753 inventory_summary AS (
754 SELECT SUM(low_stock_count) AS total_low_stock, SUM(product_count) AS total_products
755 FROM kpi.inventory_health
756 )
757 SELECT
758 rs.total_revenue, rs.total_profit, rs.total_orders,
759 cs.total_customers, cs.active_customers, cs.repeat_customers,
760 tt.territory_name AS top_territory_name, tt.lifetime_revenue AS top_territory_revenue,
761 tp.product_name AS top_product_name, tp.total_revenue AS top_product_revenue,
762 tsp.employee_name AS top_salesperson_name, tsp.total_revenue AS top_salesperson_revenue,
763 inv.total_low_stock, inv.total_products
764 FROM revenue_summary rs, customer_summary cs, top_territory tt, top_product tp, top_salesperson tsp, inventory_summary inv;
765 -- VERIFICATION
766 SELECT * FROM kpi.executive_summary;

```

	total_revenue numeric	total_profit numeric	total_orders numeric	total_customers bigint	active_customers bigint	repeat_customers bigint	top_territory_name character varying (50)	top_territory_revenue numeric	top_product_name character varying (50)	top_product_revenue numeric	top_salesperson_name text	top_salesperson_revenue numeric	total_low_stock numeric	total_products numeric
1	109846380.90	12557724.05	31465	19820	19119	7470	Southwest	24184609.96	Mountain-200 Black, 38	4400593.46	Linda Mitchell	10367007.33	80	504

12. Conclusion

This project delivers a complete, tested analytics layer on top of the AdventureWorks operational database: ten domain tables spanning five business areas, twenty seven business metric views, one executive summary view, an enforced bonus challenge proving the analytics layer is self-sufficient for future dashboards, and a Python notebook that turns all of it into visualizations and executive recommendations. Every stage of the pipeline was tested against a live copy of the database before being handed off, rather than written and assumed to work, so the deliverables in this submission reflect what actually runs, not just what was intended to run.